

遙測與空間資訊之分析與應用 - Week 7

國家災害防救科技中心 資訊組

台灣大學土木工程學系 教授

蘇文瑞

wraysu@ntu.edu.tw

The Captain's Intuition: Geometry is not Logic

Week Recap Arc:

W3: Where — How far from the river? (ARIA v1.0)

W4: How Much — How steep is the terrain? (ARIA v2.0)

W5: When — How heavy is the rain right now? (ARIA v3.0)

W6: What-If — What about areas with no stations? (ARIA v3.5)

W7: Can We Reach? — Can we actually get there? (ARIA v4.0)

Core Shift:

Traditional GIS: Points and Lines — geometric objects.

Network Science: Nodes and Edges — logical connections.

The Glass Box Question:

"Why can't a computer drive along a LineString?"

Answer: LineString has no Connectivity or Topology.

Notebook Cell [1]: Load environment.

Network Science 101: Nodes & Edges

Node: intersection, junction, start/end point.

Edge: road segment connecting two Nodes.

Attributes on Edges:

length: segment length (meters).

maxspeed: speed limit (km/h).

highway: road type (primary, secondary, residential...).

Directed vs. Undirected:

One-way street = Directed Edge.

Two-way road = Two Directed Edges (or one Undirected).

OSMnx default: MultiDiGraph (directed multigraph).

OSMnx: Real Road Networks in Python

OpenStreetMap (OSM): World's largest open map database.

OSMnx: Python library — one line fetches real road networks.

Core API:

```
G = ox.graph_from_address('Hualien', dist=5000, network_type='drive')
```

```
G = ox.project_graph(G, to_crs='EPSG:3826') # meters
```

network_type options:

'drive': vehicle roads (most common).

'walk': pedestrian paths.

'bike': bicycle paths.

'all': all road types.

Note: First fetch may take 10-30 seconds (downloads from OSM server).

Notebook Cell [2]: Fetch NTU campus road network.

Weighting: The Logic of Travel

Three weighting strategies:

1. Distance: shortest path (meters) — not always fastest.
2. Time: fastest path (seconds) = Length / Speed Limit.
3. Disaster: disaster-time path — add flood, congestion, closure factors.

Basic time weight:

```
for u, v, k, data in G.edges(data=True, keys=True):  
    speed = data.get('maxspeed', 40) # default 40 km/h  
    data['travel_time'] = data['length'] / (speed / 3.6) # seconds
```

The Disaster Shift:

How do we put W5 rainfall intensity and W4 slope risk into edge weights?

This is today's core question.

Notebook Cell [3]: Calculate basic travel time.

Dijkstra's Algorithm: The Glass Box

Problem: What's the shortest (or fastest) path from A to B?

Dijkstra's approach:

1. Start from origin, mark distance = 0.
2. Explore all neighbors, record cumulative cost.
3. Pick the neighbor with minimum cost.
4. Repeat until destination is reached.

NetworkX implementation:

```
route = nx.shortest_path(G, source=orig, target=dest, weight='travel_time')  
cost = nx.shortest_path_length(G, source=orig, target=dest, weight='travel_time')
```

Glass Box Point: Dijkstra guarantees optimality (not greedy), but requires weight ≥ 0 .

Notebook Cell [4]: Shortest path demo + visualization.

Betweenness Centrality: Finding the Bottlenecks

Problem: Which intersections are most 'important'? If one fails, what's the biggest impact?

Betweenness Centrality:

Counts how many shortest paths pass through each node.

Higher value = more paths must go through this intersection = BOTTLENECK.

Disaster significance:

High-centrality intersections flooded/collapsed = massive cascade failure.

These are the 'traffic lifelines' that need priority monitoring.

NetworkX implementation:

```
centrality = nx.betweenness centrality(G, weight='length')
```

```
top5 = sorted(centrality, key=centrality.get, reverse=True)[:5]
```

Complexity: $O(n*m)$. Hualien City ~3000 nodes = seconds. Full county = 1-2 min.

Notebook Cell [5]: Centrality calculation + visualization.

Lab 1: Bottleneck Identification (25 min)

Task: Fetch NTU campus 1.5km road network.

Calculate betweenness centrality.

Find Top 5 bottleneck intersections.

Visualize with size-coded nodes.

Teacher Patrol Checklist:

`ox.project_graph()` done? (must be EPSG:3826 for meter distances)

`weight='length'` specified? (without `weight` = hop count)

Top 5 marked with different colors/sizes?

Advanced Challenge:

Overlay Top 5 nodes with W3 shelter locations — any overlap?

Notebook Cells [6]-[7]: Bottleneck analysis + histogram.

The 'Broken Edge': Simulating Road Closure

Scenario: Typhoon Fung-wong causes road closure on Suhua Highway.

Simulation strategy:

1. Find target edge: (u, v, key).
2. `G.remove_edge(u, v, key=key)` — simulate closure.
3. Recalculate shortest path — observe detour effect.

Resilience Score (Detour Ratio):

Detour Ratio = post-disaster travel time / pre-disaster travel time

Ratio = 1.0: no impact.

Ratio = 2.0: travel time doubled.

Ratio = infinity: completely disconnected (island!).

Code Pattern:

```
try: t_after = nx.shortest_path_length(G_broken, orig, dest, weight='travel_time')
except nx.NetworkXNoPath: detour_ratio = float('inf') # Island!
```

Notebook Cell [8]: Road closure simulation.

Isochrones: The Truth About Accessibility

Isochrone: From one point, the reachable area within a fixed time.

5-min isochrone: ambulance coverage in 5 minutes.

10-min isochrone: fire truck coverage in 10 minutes.

Calculation logic:

```
travel_times = nx.single_source_dijkstra_path_length(G, source, weight='travel_time')
```

```
reachable = [node for node, t in travel_times.items() if t <= 300] # 5 min
```

Visualization: Use `alphashape` or `concave_hull` to form polygon.

Area calculation: `isochrone_polygon.area` (EPSG:3826 = m²).

Key insight: Isochrones are NOT circles.

They follow roads, like octopus tentacles.

Notebook Cell [9]: Isochrone calculation.

The Buffer Lie vs. Isochrone Truth

Buffer (Week 3):

Assumes 'straight-line distance = actual distance'.

Draws a perfect circle.

Problem: ignores roads, terrain, one-way streets.

Isochrone (Week 7):

Uses actual road network topology.

Draws an irregular polygon.

Reflects real accessibility.

Shocking comparison:

Buffer 5km: covers 78.5 km² (perfect circle).

Isochrone 5min: might only cover 12 km² (follows roads).

Gap: over 6x overestimate!

Disaster significance: Using Buffer to estimate shelter coverage = severe overestimation.

Dynamic Weighting: Rain-Weighted Routing

Core formula (congestion factor):

$\text{travel_time_adj} = \text{length} / (\text{speed} * (1 - \text{congestion_factor}))$

cf = 0: normal travel.

cf = 0.3: travel time increases ~43%.

cf = 0.6: travel time increases 2.5x.

cf = 0.9: travel time increases 10x (nearly impassable).

Rainfall to congestion mapping (threshold method):

Hourly rain < 10mm: cf = 0.0 (normal).

Hourly rain 10-40mm: cf = 0.3 (slightly slow).

Hourly rain 40-80mm: cf = 0.6 (severely delayed).

Hourly rain > 80mm: cf = 0.9 (nearly blocked).

Suao during Typhoon Fung-wong: 130.5mm/hr → cf=0.9 → speed drops to 10%.

Notebook Cell [10]: Dynamic weight function.

Lab 2: Dynamic Routing & Isochrone Shrinkage (30 min)

Task:

1. Fetch Hualien City road network.
2. Apply congestion_factor using Typhoon rainfall data (W5/W6).
3. Calculate pre-disaster vs. post-disaster isochrones.
4. Quantify 'isochrone area shrinkage ratio'.

Teacher Patrol Checklist:

Rainfall data CRS matches road network (EPSG:3826)?
Congestion factor grading logic correct (10/40/80mm)?
Using 'travel_time_adj' not 'travel_time' for post-disaster?
If shrinkage > 50%, ask: 'What should the commander do?'

Advanced Challenge:

Use W6 Kriging raster instead of W5 station data for precise per-segment rainfall.
Notebook Cells [11]-[13]: Dynamic routing + isochrone comparison.

Raster-to-Network Integration: The Full Pipeline

W6 Kriging output → W7 road segment weights:

1. Read kriging_rainfall.tif (continuous rainfall field).
2. For each road segment's midpoint, do raster sampling.
3. Convert rainfall value to congestion_factor.
4. Update segment's travel_time_adj.

Code pattern:

```
import rasterio
with rasterio.open('kriging_rainfall.tif') as src:
    for u, v, k, data in G.edges(data=True, keys=True):
        midpoint = ... # edge midpoint coordinates
        rain = list(src.sample([(midpoint.x, midpoint.y)]))[0][0]
        data['congestion'] = rain_to_congestion(rain)
```

Advantage: Not limited to stations — Kriging fills the gaps.

Notebook Cell [14]: Raster-to-Network (advanced).

GraphML: Saving Network Data

Problem: Fetching from OSM every time is slow (30+ seconds).

Solution: Save as .graphml file.

```
ox.save_graphml(G, 'hualien_network.graphml')
```

```
G = ox.load_graphml('hualien_network.graphml')
```

GraphML format:

XML-based, cross-platform compatible.

Preserves all node/edge attributes.

QGIS, Gephi can also open it.

Pro Rule:

Network data is large (Hualien City ~5MB).

Each OSM fetch = 30 sec wait + server load.

After saving = 1 second to load.

Your repo should have a data/ folder with .graphml files.

Notebook Cell [15]: GraphML save/load.

AI Advisor: LLM-Assisted Logistics Strategy

Scenario: Isochrone analysis shows south Hualien is isolated.

Input to LLM (ChatGPT / Gemini / Claude / ...):

Pre-disaster isochrone: 45.2 km².

Post-disaster isochrone: 12.8 km² (shrinkage 72%).

Max Detour Ratio: 3.5x (Xiulin → Hualien City).

Isolated shelters: 3 locations.

Prompt specification:

'You are a traffic advisor for Hualien disaster command center.'

'Based on the following network analysis results: {data_summary}'

'Provide 3 recommendations: priority road repair, alternative rescue, resource allocation.'

AI Advisor is NOT making decisions —

it translates numbers into language commanders can read.

You provide data, AI provides narrative, commander makes the call.

Notebook Cell [16]: AI logistics advisor (Bonus).

ARIA v4.0: The Complete System

ARIA Evolution:

v1.0 (W3): River buffer + shelters — How far from the river?

v2.0 (W4): + DEM terrain analysis — How steep is the terrain?

v3.0 (W5): + Real-time rainfall monitoring — How heavy is the rain?

v3.5 (W6): + Kriging spatial prediction — What about unmonitored areas?

v4.0 (W7): + Network accessibility — Can we actually reach them?

v4.0 = Complete 'detect → predict → accessibility' pipeline.

Final Project:

Build on v4.0, choose your own disaster scenario and extend.

Infrastructure First: Code Patterns

Week 7 Core Pattern:

Fetch Road Network → Build Graph → Set Weights
→ Shortest Path / Centrality / Isochrone
→ Simulate Disruption → Compare Before/After
→ Export GraphML + Report

Applicable scenarios:

Urban flood evacuation route planning.
Post-earthquake road damage assessment.
Rural area medical accessibility analysis.
Post-disaster supply delivery route optimization.

When to use Network Analysis:

Need to consider 'how to get there' not just 'how far'.
Need to simulate 'road closure' or 'congestion' impact on accessibility.
Need real travel time, not straight-line distance.

Final Project Progress Tracking

This Week's Tasks (groups already formed + proposals submitted):

Confirm all groups have received feedback on proposals.

Each group reports: 'Did you find your data source? Any difficulties?'

Network Analysis inspiration: Does any group want to add W7 analysis to their project?

Timeline Reminder:

W11: Mid-term progress report (informal 5-min check-in).

W12-15: Course continues + project parallel development.

W16: Final presentation.

Resource Reminder:

LLM API pattern: demonstrated today (use your preferred AI tool).

GraphML pattern: directly usable in projects.

Week 7 Reflection

"A risk map tells you what is broken.

A network analysis tells you if you can still save someone."

Reflection Questions:

If navigation AI says this road is fastest, but you know it floods often,
how would you teach your AI Agent to incorporate 'local knowledge'?
Who should bridge the gap between digital twins and the real world?

Assignment: ARIA v4.0 — Network Integration Version

Road network extraction + bottleneck analysis + dynamic accessibility + AI strategy report.

Next Week: W8 — Spectral Interpretation: from ground view to satellite view.

Key takeaway: Buffer assumes you can fly. Isochrone is the real walking result.
From now on, any 'accessibility' analysis should use road networks, not circles.